



Digital Integrated Circuit Design

Lecture-2: Review of Verilog HDL - Supplement

Chih-Feng Wu (吳志峰), PhD.

***Department of Electronics Engineering, Chang Gung University,
Taoyuan, Taiwan (ROC)***

Mar. 27, 2026

Chih-Feng Wu



Outline

- ☐ **Useful Verilog RTL vs. Physical HW**
- ☐ **Propagation Delay**
- ☐ **Finite-State Machine (FSM)**
- ☐ **DesignWare Library**

Chih-Feng Wu

Outline

- ❑ Useful Verilog RTL vs. Physical HW
- ❑ Propagation Delay
- ❑ Finite-State Machine (FSM)
- ❑ DesignWare Library

Multiplexer

❑ 2×1MUX

■ Type-1

```
`define N 8  
  
wire SEL;  
wire [`N-1:0] A, B, MOut;  
  
assign MOut = SEL ? A : B;
```

■ Type-2

```
wire SEL;  
wire [`N-1:0] A, B;  
reg [`N-1:0] MOut;  
  
always @(A, B)  
    if (SEL)  
        MOut = A;  
    else  
        MOut = B;
```

Multiplexer (cont'd)

□ 4×1MUX

```
`define M 2
`define N 8

wire [`M-1:0] SEL;
wire [`N-1:0] A, B, C, D;
reg  [`N-1:0] MOut;

always @(SEL or A or B or C or D)
begin
    case(SEL)
        2'b00: MOut = A;
        2'b01: MOut = B;
        2'b10: MOut = C;
        2'b11: MOut = D;
        default: MOut = A;
    endcase
end
```

Chih-Feng Wu

5

Multiplexer (cont'd)

□ 4×1MUX with don't case x

```
`define M 2
`define N 8

wire [`M-1:0] SEL;
wire [`N-1:0] A, B, C;
reg  [`N-1:0] MOut;

always @(SEL or A or B or C)
begin
    case(SEL)
        2'b00: MOut = A;
        2'b01: MOut = B;
        2'b1x: MOut = C;
        default: MOut = C;
    endcase
end
```

Chih-Feng Wu

6

Encoder

□ Priority Encoder using if-else

```
`define M 2
`define N 8
// :
wire [`M-1:0] SEL;
wire [`N-1:0] A, B, C, D;
reg [`N-1:0] MOut;

always @(SEL or A or B or C or D)
begin
    if (SEL == 2'b00)
        MOut = A;
    else if (SEL == 2'b01)
        MOut = B;
    else if (SEL == 2'b10)
        MOut = C;
    else
        MOut = D;
end
```

Chih-Feng Wu

7

Decoder

□ Decoder using case

```
`define N 8

wire [`M-1:0] AIn;
reg [`N-1:0] YOut;

always @(AIn)
case (AIn)
    3'b000: YOut = (`N)'b0000_0001;
    3'b001: YOut = (`N)'b0000_0010;
    3'b010: YOut = (`N)'b0000_0100;
    3'b011: YOut = (`N)'b0000_1000;
    3'b100: YOut = (`N)'b0001_0000;
    3'b101: YOut = (`N)'b0010_0000;
    3'b110: YOut = (`N)'b0100_0000;
    3'b111: YOut = (`N)'b1000_0000;
endcase
```

Chih-Feng Wu

8

Register (D-FF)

- ❑ Rising edge FF (1-bit) with **synchronous** reset

```
wire DIn;
reg Q;
wire CLK, RESET_;

always @(posedge CLK)
  if (!RESET_)
    Q <= 1'b0; // 0;
  else
    Q <= DIn;
```

- ❑ Rising edge FF (1-bit) with **asynchronous** reset

```
wire DIn;
reg Q;
wire CLK, RESET_;

always @(posedge CLK or negedge RESET_)
  if (!RESET_)
    Q <= 1'b0; // 0;
  else
    Q <= DIn;
```

9

Register (cont'd)

- ❑ Rising edge FFs with **synchronous** reset

```
`define N 8

wire [`N-1:0] DIn;
reg [`N-1:0] Q;
wire CLK, RESET_;

always @(posedge CLK)
  if (!RESET_)
    Q <= ('N)'d0; // 0;
  else
    Q <= DIn;
```

- ❑ Rising edge FFs with **asynchronous** reset

```
`define N 8

wire [`N-1:0] DIn;
reg [`N-1:0] Q;
wire CLK, RESET_;

always @(posedge CLK or negedge RESET_)
  if (!RESET_)
    Q <= ('N)'d0; // 0;
  else
    Q <= DIn;
```

Register (cont'd)

- ❑ Rising edge FF with asynchronous preset

```
wire DIn;
reg Q;
wire CLK, PRESET_;

always @(posedge CLK or negedge PRESET_)
  if (!PRESET_)
    Q <= 1'b1;
  else
    Q <= DIn;
```

Register (cont'd)

- ❑ Rising edge FFs with asynchronous reset and clock enable

```
`define N 8

wire [`N-1:0] DIn;
reg [`N-1:0] Q;
wire ENAB;
wire CLK, RESET_;

always @(posedge CLK or negedge RESET_)
  if (!RESET_)
    Q <= `N'b0000_0000;
  else if (ENAB)
    Q <= DIn;
```

Counter

- ❑ 8-bit Counter with enable and asynchronous reset (Rising edge FF)

```
define N 8

wire ENAB;
reg  [`N-1:0] CNT;
wire CLK, RESET_;

always @(posedge CLK or negedge RESET_)
  if (!RESET_)
    CNT <= (`N)'d0;
  else if (ENAB)
    CNT <= CNT + 1'b1;

always @(posedge CLK or negedge RESET_)
  if (!RESET_)
    CNT <= (`N)'b0;
  else
    CNT <= CNT + 1'b1;
```

Chih-Feng Wu

13

Counter (cont'd)

- ❑ 8-bit Counter with load and asynchronous reset (Rising edge FF)

```
define N 8

wire [`N-1:0] DIn;
reg  [`N-1:0] CNT;
wire CLK, RESET_;

always @(posedge CLK or negedge RESET_)
  if (!RESET_)
    CNT <= (`N)'d0;
  else if (LOAD)
    CNT <= DIn;
  else
    CNT <= CNT + 1'b1;

always @(posedge CLK or negedge RESET_)
  if (!RESET_)
    CNT <= (`N)'b0;
  else if (LOAD)
    CNT <= DIn;
  else if (ENAB)
    CNT <= CNT + 1'b1;
```

eng Wu

14

Arithmetic

□ Arithmetic operators

```
`define N 8

wire [`N-1:0] A, B;
reg  [`N-1:0] O_ADD, O_SUB, O_MUL, O_DIV;

always @(A or B)
begin
    O_ADD = A + B;
    O_SUB = A - B;
    O_MUL = A * B;
    O_DIV = A / B;
end
```

Relational Operators

□ Relational Operators (>, <, >=, <=, ==, !=)

```
`define N 8

wire [`N-1:0] A, B;
reg  ROut;

always @(A or B)
begin
    if (A > B) // (A < B), (A >= B), (A <= B), (A == B), (A != B)
        ROut = 1;
    else
        ROut = 0;
end

// -----
wire ROut;
// -----
assign ROut = (A > B) ? 1 : 0;
// -----
```


Shift Operators

□ Shift Operators (>>, <<)

```
`define N 8
`define M 4

wire [`N-1:0] Din;
reg  [`N-1:0] OSL, OSR;

always @(DIn)
begin
    OSL = Din << (`N-`M);
    OSR = Din >> `M;
end
```

Important Case Study

□ Multiple arithmetic operations

```
`define N 8
`define M 4

wire SEL;
wire [`N-1:0] A, B, C, D;
reg  [`N-1:0] SUM;

always @(A or B)
if (SEL)
    SUM = A + B;
else
    SUM = C + D;
:
// -----
wire [`N-1:0] SUM;
:
assign SUM = (SEL) ? A + B : C + D;
:
// -----
```

Important Case Study (cont'd)

❑ Multiple multiplexers with single arithmetic operation

```

wire SEL;
wire [N-1:0] A, B, C, D;
reg [N-1:0] TMP1, TMP2;
wire [N-1:0] SUM;

always @(A or B)
  if (SEL)
    begin
      TMP1 = A;
      TMP2 = B;
    end
  else
    begin
      TMP1 = C;
      TMP2 = D;
    end

assign SUM = TMP1 + TMP2;
:
// -----
wire [N-1:0] TMP1, TMP2;
:
assign TMP1 = SEL ? A : C;
assign TMP2 = SEL ? B : D;
assign SUM = TMP1 + TMP2;
:
// -----

```

Chih-Feng Wu

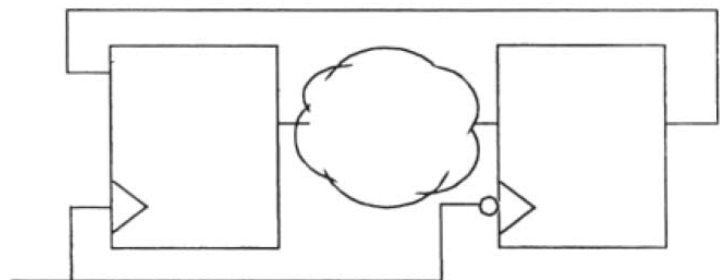
19

Important Case Study (cont'd)

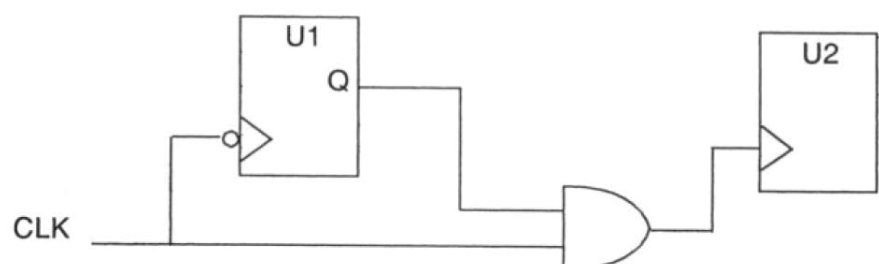
❑ Do NOT use multiple CLKs in your single design

❑ Bad clock schemes

- Mixed clock edges in single design



- Gating clock: limited testability and skew problems



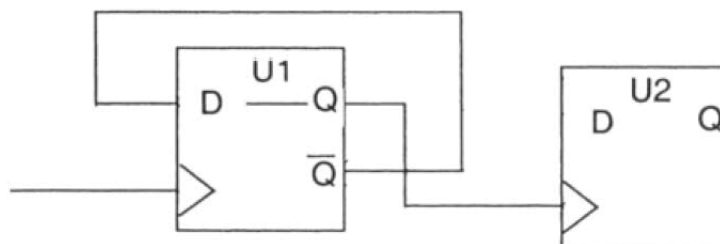
Chih-Feng Wu

20

Important Case Study (cont'd)

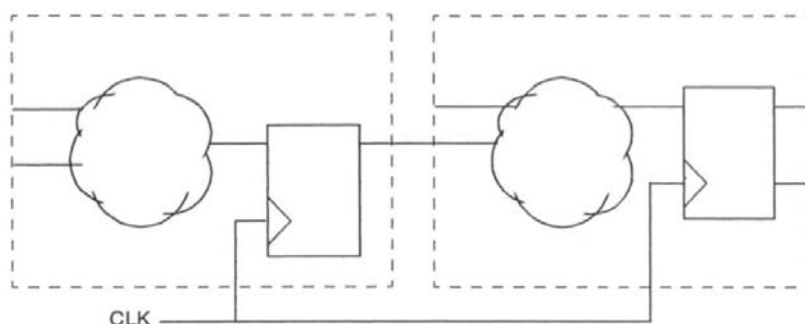
❑ Bad clock schemes

■ Internal gating clock



❑ Ideal clock scheme

■ Single clock in single design

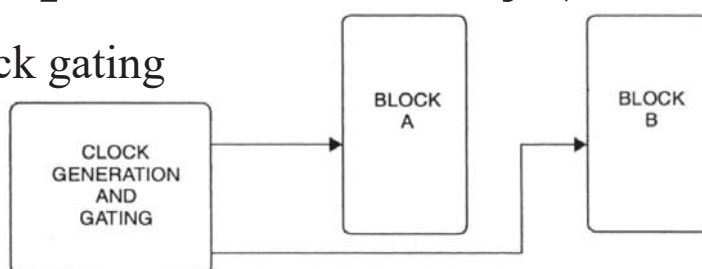


Chih-Feng Wu

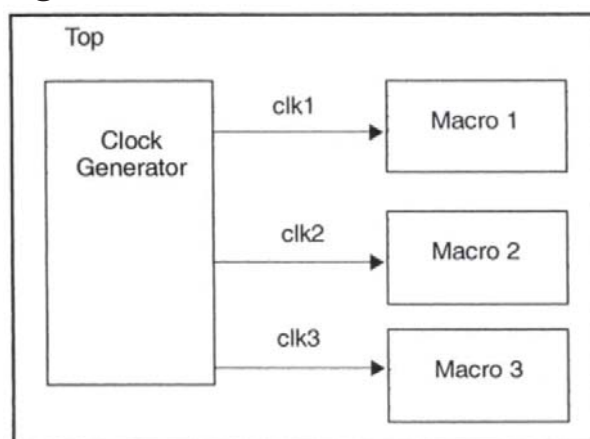
21

Important Case Study (cont'd)

❑ Block clock gating



❑ Isolation clock generation at TOP

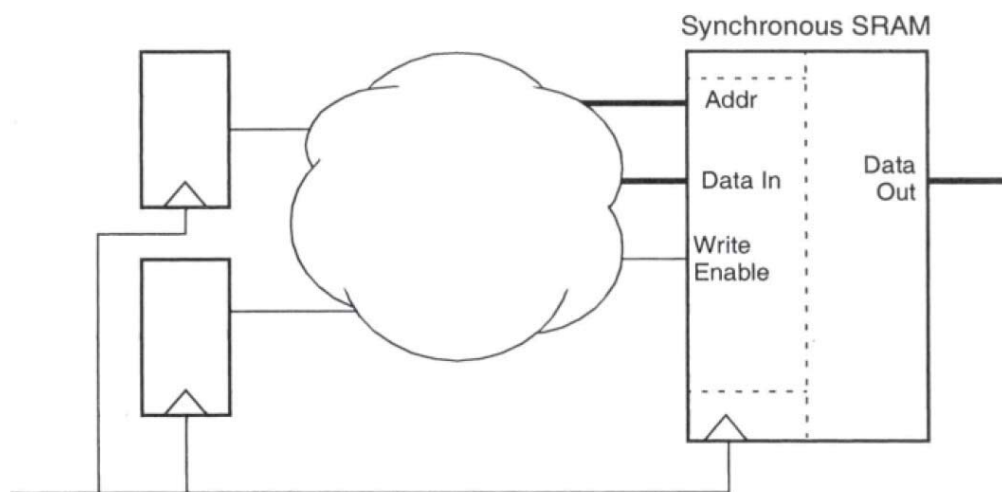


Chih-Feng Wu

22

Important Case Study (cont'd)

☐ **Synchronous** memory interface



Chih-Feng Wu

23

Outline

- ☐ Useful Verilog RTL vs. Physical HW
- ☐ Propagation Delay
- ☒ **Finite-State Machine (FSM)**
- ☐ DesignWare Library

Chih-Feng Wu

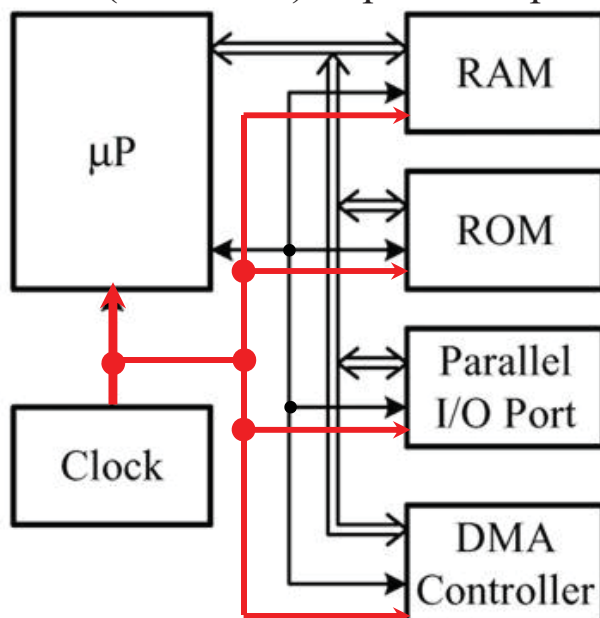
24

Finite-State Machine (FSM)

- ❑ Finite-state machine (FSM)
 - Definition
 - Modeling style

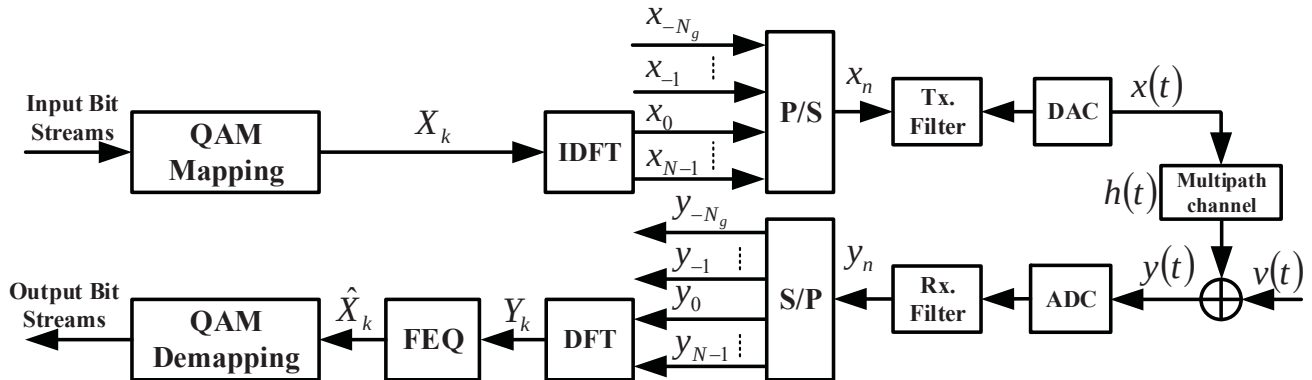
General Digital System

- ❑ Why does a general digital system has to use FSM?
 - Control the blocks (or circuits) to perform specific sequences of operations



IEEE 802.11a OFDM System

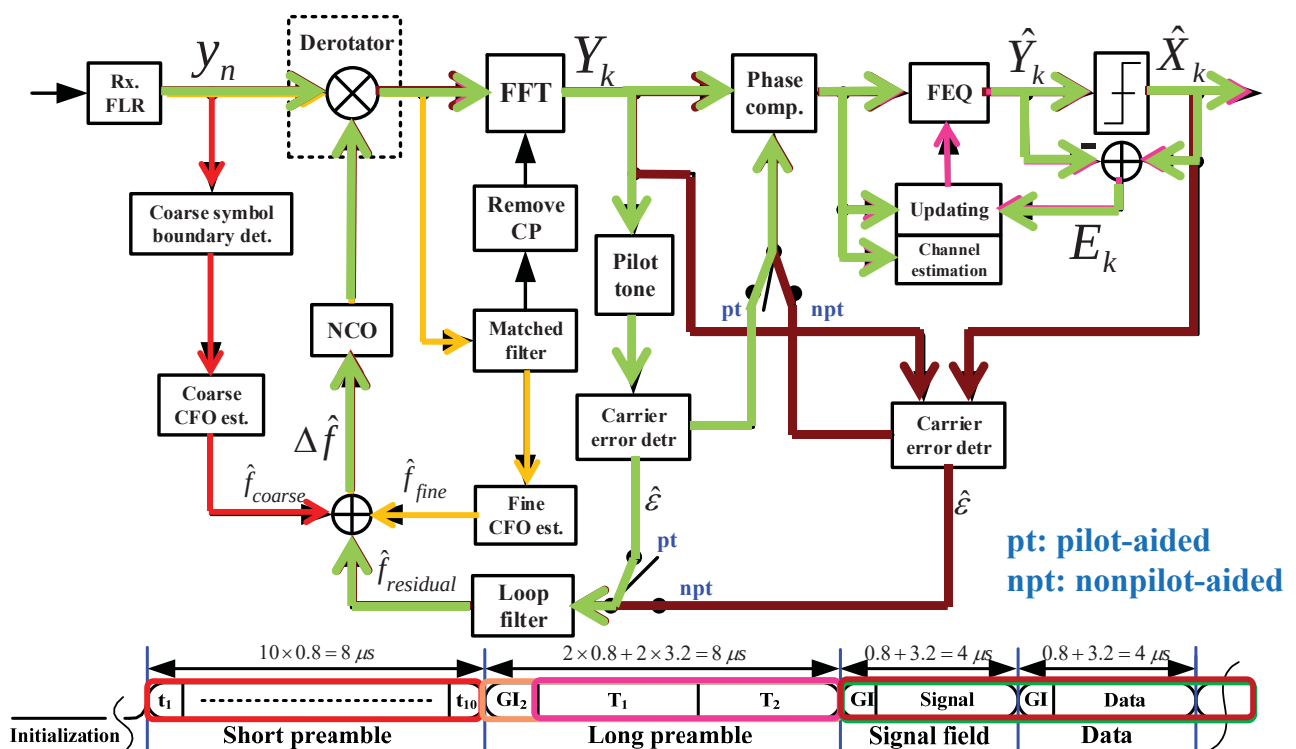
Block diagram of OFDM transceiver



Chih-Feng Wu

27

Receiver Template for IEEE 802.11a WLAN

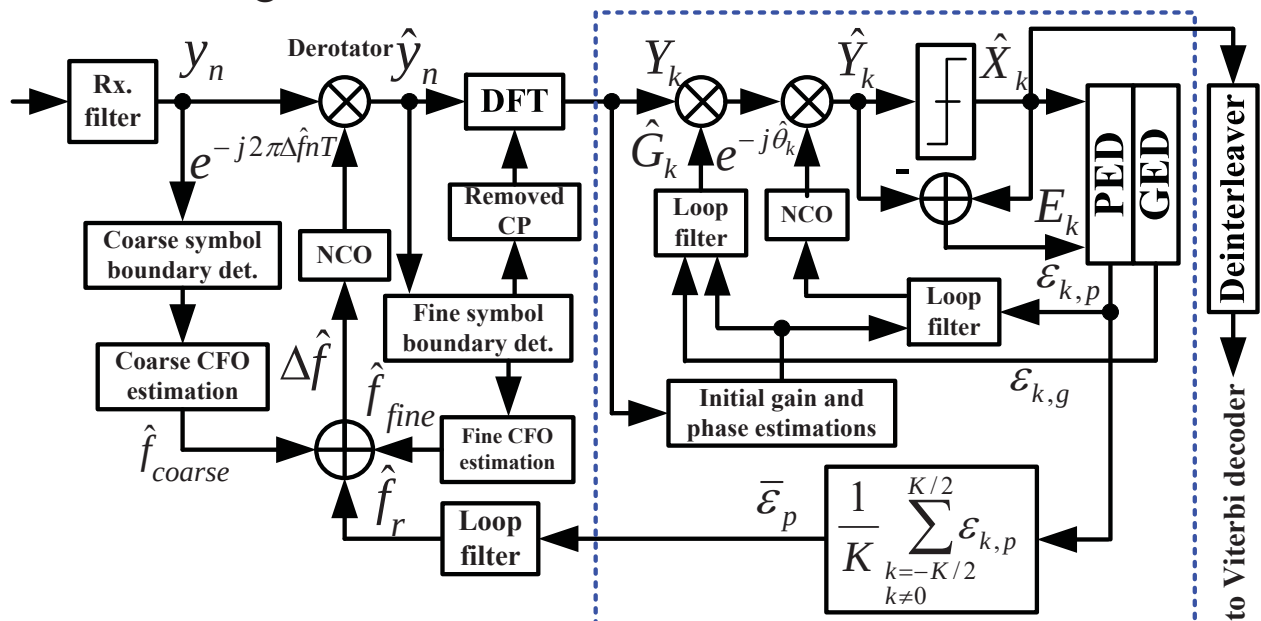


Chih-Feng Wu

28

Joint Algorithm for IEEE 802.11a WLAN Receiver

□ Block diagram

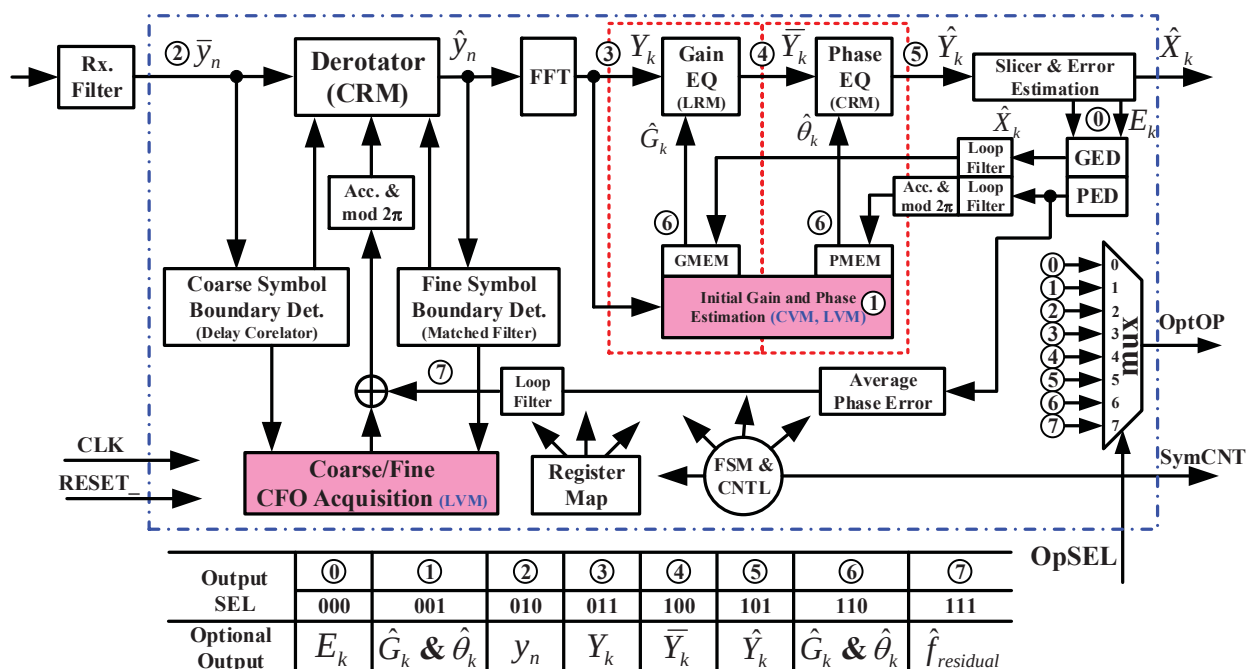


Chih-Feng Wu

29

Joint Algorithm for IEEE 802.11a WLAN Receiver (cont'd)

❑ Hardware architecture



Chih-Feng Wu

30

Finite-State Machine (FSM)

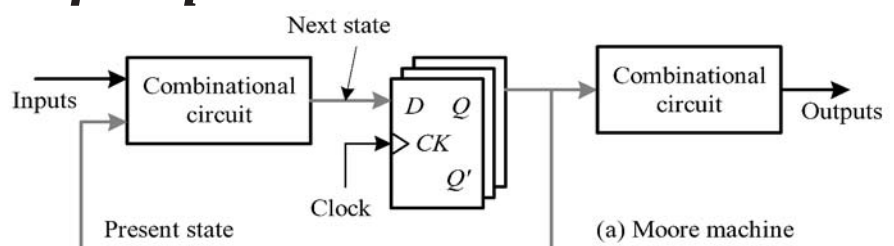
□ A finite-state machine (FSM)

$$M = (I, O, S, \delta, \lambda)$$

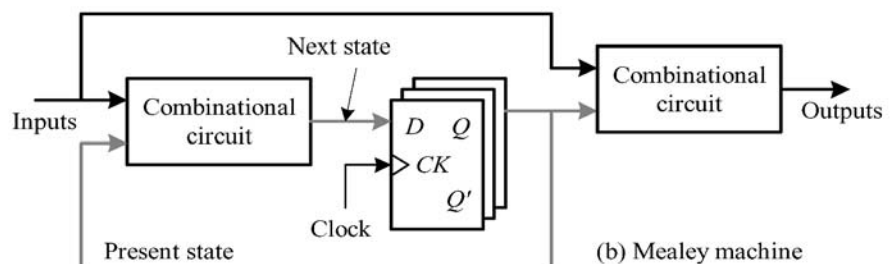
- I , O , and S are finite, nonempty sets of **input**, **output**, and **state** symbols
- State transition function $\delta: I \times S \rightarrow S$
 - Determine by both input and state symbol
- Output function λ :
 1. $\lambda: I \times S \rightarrow O$ (**Mealy** machine)
 - Output function determined by both present input and state
 2. $\lambda: S \rightarrow O$ (**Moore** machine)
 - Output function only determined by present state

Type of Sequential Circuit

□ Moore machine



□ Mealy machine



□ Components of FSM

- **Combinational** circuits (output circuits)
- **State circuits** (current/next state) composed of sequential circuit

State Encoding of State Machine

❑ Common FSM encoding options

- One-hot code: one of the state registers to 1 at each state
- **Binary** code: assign binary code to state registers according to the state sequence
- **Gray** code: assign Gray code to state registers according to the state sequence
- Random code: assign random code to state registers for each state

State	Binary	Gray	One hot
A	00	00	1000
B	01	01	0100
C	10	11	0010
D	11	10	0001

❑ One-hot encoding

- Usually used in **FPGA-based** designs since a lot of registers available for use in these devices

Chih-Feng Wu

33

FSM Modeling Style

❑ Two major implementation issues of a FSM

- Declaration and update of state register
- Computation of both next-state and output function

❑ State register

- One register
- Two register: Current (present) state and Next state

❑ Output and next state function

- Continuous assignment
- Function
- **always** block
- a combination of the above

Chih-Feng Wu

34

Modeling Style

❑ Style 1: one state register

■ part 1: initialize and update the state register

```
always @(posedge clk or negedge reset_n)
  if (!reset_n) state <= initial_state;
  else          state <= next_state (state, x) ;
```

■ part 2: determine next state using function

```
function next_state (input present_state, x)
  case (present_state ) ...
  endcase
endcase endfunction
```

■ part 3: determine output function

```
always @(state or x) case (state) ... endcase or
assignment statements
```

Chih-Feng Wu

35

Modeling Style (cont'd)

❑ Style 2: two state registers

■ part 1: initialize and update the state register

```
always @(posedge clk or negedge reset_n)
  if (!reset_n) present_state <= initial_state;
  else          present_state <= next_state;
```

■ part 2: determine next state

```
always @(present_state or x)
  case (present_state ) ... endcase or
assignment statements
```

■ part 3: determine output function

```
always @(present_state or x)
  case (present_state ) ... endcase or
assignment statements
```

Chih-Feng Wu

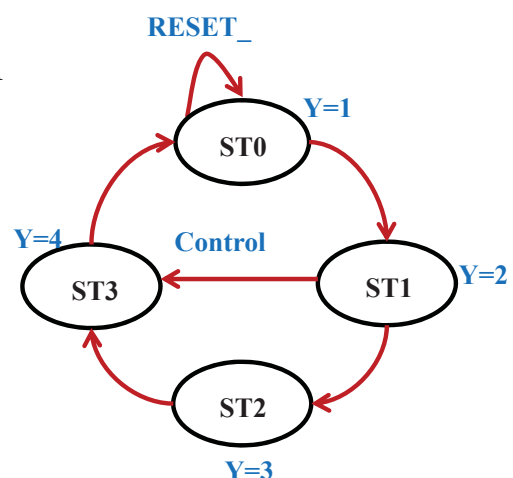
36

Coding Style (cont'd)

- ❑ State name
 - Described by using the “**parameter**”
- ❑ Combinational logic for computing the next state
 - Separated from the state registers, i.e., using its own “**always**” block
- ❑ Combinational logic of the next state function
 - Implemented with a “**case**” statement

Four State FSM

- ❑ FSM with asynchronous RESET_
 - State: ST0, ST1, ST2, ST3
 - Control signal: RESET_, Control
 - Output signal: Y



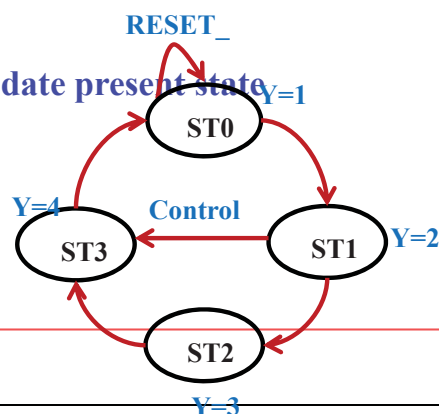
Example A1 – Separate CS, NS and OL

```

module FSM_A1(CLK, RESET_, Control, Y);
  input CLK, RESET_, Control;
  output [2:0] Y;
  // local declaration
  parameter [1:0] ST0 = 2'b00, ST1 = 2'b01, ST2 = 2'b10, ST3 = 2'b11;
  reg [1:0] Current_ST, Next_ST; // present state and next state

  // part 1: initialize to state and update state register
  always @(posedge CLK or negedge RESET_)
    if (! RESET_) Current_ST <= ST0;
    else          Current_ST <= Next_ST; // update present state
  // part 2: determine next state
  always @(Control or Current_ST)
    case (Current_ST)
      ST0: Next_ST = ST1;
      : (see next page)
    endcase

```



Chih-Feng Wu

39

Example A1 (cont'd)

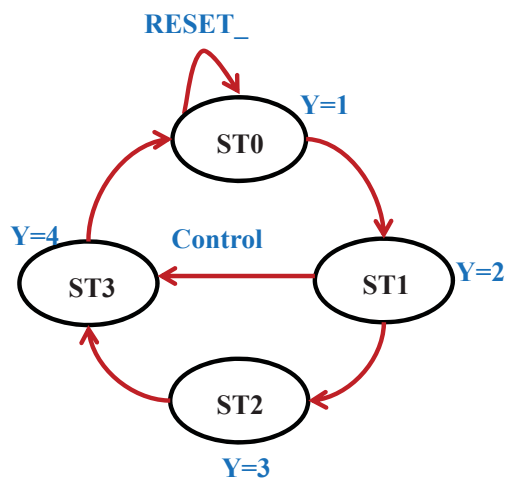
```

      ST1: if (Control) Next_ST = ST3;
           else       Next_ST = ST2;

      ST2: Next_ST = ST3;

      ST3: Next_ST = ST0;
    endcase
  // part 3: generate output logic
  always @(Current_ST)
    begin
      case (Current_ST)
        ST0: Y=1;
        ST1: Y=2;
        ST2: Y=3;
        ST3: Y=4;
        default: Y=1;
      endcase
    end

```



Chih-Feng Wu

40

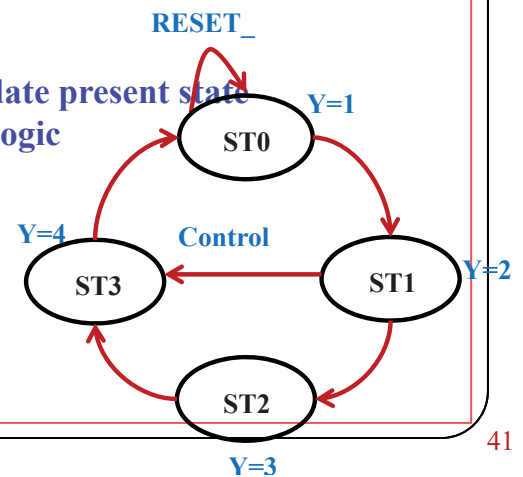
Example A2 – Combined NS and OL

```

module FSM_A2(CLK, RESET_, Control, Y);
  input  CLK, RESET_, Control;
  output [2:0] Y;
  // local declaration
  parameter [1:0] ST0 = 2'b00, ST1 = 2'b01, ST2= 2'b10, ST3= 2'b11;
  reg [1:0] Current_ST, Next_ST; // present state and next state

  // part 1: initialize to state and update state register
  always @(posedge CLK or negedge RESET_)
    if (! RESET_) Current_ST <= ST0;
    else          Current_ST <= Next_ST; // update present state
  // part 2: determine next state and generate output logic
  always @(Control or Current_ST)
    case (Current_ST)
      ST0: begin
        Y=1; Next_ST = ST1;
      end
      ST1: begin
        Y=2; Next_ST = ST2;
      end
      ST2: begin
        Y=3; Next_ST = ST3;
      end
      ST3: begin
        Y=4; Next_ST = ST0;
      end
    endcase
endmodule
: (see next page)

```



Chih-Feng Wu

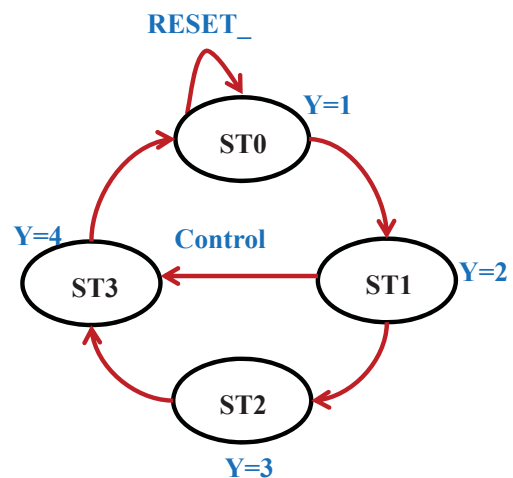
41

Example A2 (cont'd)

```

      ST1: begin
        Y=2;
        if (Control) Next_ST = ST2;
        else          Next_ST = ST3;
      end
      ST2: begin
        Y=3; Next_ST = ST3;
      end
      ST3: begin
        Y=4; Next_ST = ST0;
      end
      default: begin
        Y=1; Next_ST = ST0;
      end
    endcase
endmodule

```



Chih-Feng Wu

42



Outline

- ❑ Useful Verilog RTL vs. Physical HW
- ❑ Propagation Delay
- ❑ Finite-State Machine (FSM)
- ❑ **DesignWare Library**



DesignWare Library

DesignWare Library - Datapath and Building Block IP

The DesignWare Library's Datapath and Building Block IP is a collection of reusable intellectual property blocks that are tightly integrated into the Synopsys synthesis environment. Using the DesignWare Library's Datapath and Building Block IP allows transparent, high-level optimization of performance during synthesis. The large availability of IP components enables design reuse and significantly improves productivity. The DesignWare Library's Datapath and Building Block IP consists of:

Application Specific: Control Logic	
DW_arb_2t	Two-Tier Arbiter with Dynamic/Fair-Among-Equal Scheme
DW_arb_dp	Arbiter with Dynamic Priority Scheme
DW_arb_fcfs	Arbiter with First-Come-First-Served Priority Scheme
DW_arb_rr	Arbiter with Round Robin Priority Scheme
DW_arb_sp	Arbiter with Static Priority Scheme
Datapath: Arithmetic Components	
DW01_absval	Absolute Value
DW01_add	Adder
DW01_addsub	Adder-Subtractor
DW01_ash	Arithmetic Shifter
DW01_bsh	Barrel Shifter
DW01_cmp2	2-Function Comparator
DW01_cmp6	6-Function Comparator
DW01_csa	Carry Save Adder

SEARCH TOOLS

Search for IP...

Analog IP Selector

Foundation IP Selector

Non-Volatile Memory IP Selector

More IP Resources

[News](#)
[Videos](#)
[Articles](#)
[Customer Successes](#)
[White Papers](#)
[Webinars](#)
[Blogs](#)
[Synopsys IP Technical Bulletin](#)
[IP Reuse Tools](#)
[IP OEM Partner Program](#)



DesignWare Library

Datapath: Arithmetic Components

DW01_absval	Absolute Value
DW01_add	Adder
DW01_addsub	Adder-Subtractor
DW01_ash	Arithmetic Shifter
DW01_bsh	Barrel Shifter
DW01_cmp2	2-Function Comparator
DW01_cmp6	6-Function Comparator
DW01_csa	Carry Save Adder
DW01_dec	Decrementer
DW01_inc	Incrementer
DW01_incdec	Incrementer-Decrementer
DW01_satrnd	Arithmetic Saturation and Rounding Logic
DW01_sub	Subtractor
DW02_mac	Multiplier-Accumulator
DW02_mult	Multiplier
DW02_multp	Partial Product Multiplier
DW02_mult_2_stage	Two-Stage Pipelined Multiplier
DW02_mult_3_stage	Three-Stage Pipelined Multiplier
DW02_mult_4_stage	Four-Stage Pipelined Multiplier
DW02_mult_5_stage	Five-Stage Pipelined Multiplier
DW02_mult_6_stage	Six-Stage Pipelined Multiplier
DW02_prod_sum	Generalized Sum of Products
DW02_prod_sum1	Multiplier-Adder

[News](#)

[Videos](#)

[Articles](#)

[Customer Successes](#)

[White Papers](#)

[Webinars](#)

[Blogs](#)

[Synopsys IP Technical Bulletin](#)

[IP Reuse Tools](#)

[IP OEM Partner Program](#)

[myDesignWare Subscriptions](#)

Chih-Feng Wu